

딥러닝 파이토치 교과서 ~6.1.4

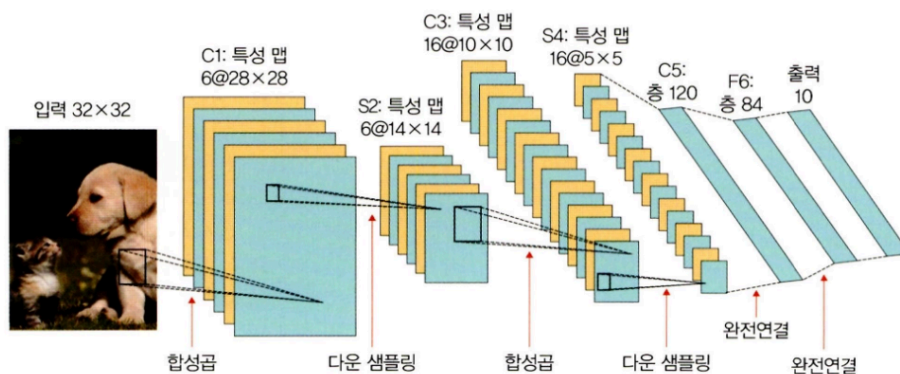
제출	제출 전
과제 유형	예습
마감일	@2025년 4월 7일
발표	☐
완료	☐
주차	5주차

6.1 이미지 분류를 위한 신경망

- 입력 데이터로 이미지를 사용한 분류는 특정 대상이 영상 내에 존재하는지 여부를 판단하는 것

6.1.1 LeNet-5

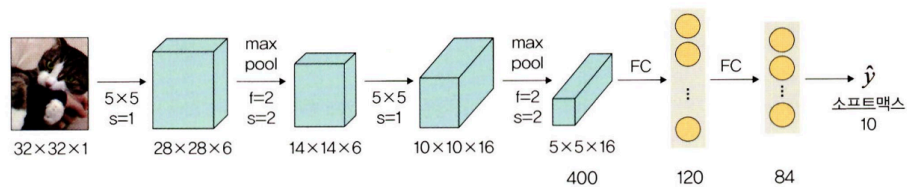
- 합성곱 신경망이라는 개념을 최초로 얀 르쿤이 개발한 구조로, 수표에 쓴 손글씨 숫자를 인식하는 딥러닝 구조
- 현재 CNN의 초석
- LeNet-5는 합성곱과 다운 샘플링 (풀링)을 반복적으로 거치며 마지막에 완전 연결층에서 분류를 수행함



- C: 합성곱층, S: Pooling, F: 완전 연결층

1. C1에서 5×5 합성곱 연산 후 28×28 의 크기의 특성맵 6 개 생성 (필터 개수: 6개)
2. S2에서 다운 샘플링하여 특성맵 크기를 14×14 로 줄입니다. (필터 x)
3. 다시 C3에서 5×5 합성곱 연산하여 10×10 크기의 연산 16개 생성 (필터 개수: 16개)
4. S4에서 다운 샘플링하여 특성 맵 크기를 16개의 5×5 로 줄인다 (필터 x)
5. C5에서 5×5 합성곱 연산하여 1×1 크기의 특성맵 120개 생성 (필터 개수: 120개)
6. F6에서 완전 연결층으로 C5의 결과를 유닛 (or node) 84개에 연결시킴 (필터 x, Dense layer)

LeNet-5 예제 구현



- 32×32 크기의 이미지에 합성곱층과 최대 풀리층이 쌍으로 두 번 적용된 후 완전 연결층을 거쳐 이미지가 분류되는 신경망

▼ 표 6-1 LeNet-5 예제 신경망 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	32×32	—	—	—
합성곱층	6	28×28	5×5	1	렐루(ReLU)
최대 풀링층	6	14×14	2×2	2	—
합성곱층	16	10×10	5×5	1	렐루(ReLU)
최대 풀링층	16	5×5	2×2	2	—
완전연결층	—	120	—	—	렐루(ReLU)
완전연결층	—	84	—	—	렐루(ReLU)
완전연결층	—	2	—	—	소프트맥스(softmax)

** 특성맵은 필터에 의해 출력되는 결과

◆ 필터 (또는 커널, weight)

- 학습 가능한 파라미터
- 크기: 예를 들어 5×5 (입력 채널 수만큼 깊이 있음)
- 입력 이미지에 슬라이딩 하면서 연산
- 하나의 필터 → 하나의 특성맵 생성

◆ 특성맵 (Feature Map)

- 필터 연산을 거쳐 나온 출력 결과
- 필터가 무엇을 감지했는지를 공간적으로 보여줌
- 여러 개의 필터를 쓰면 → 여러 개의 특성맵 생성

```
conda install -c conda forge tqdm
```

- tqdm: 진행 상태를 bar 형태로 가시화하여 보여줌

데이터셋 준비

```
# 필요한 라이브러리 호출
import torch
import torchvision
from torch.utils.data import DataLoader, Dataset
from torchvision import transforms # 이미지 변환 (전처리) 기능을 제공하는 라이브러리
from torch.autograd import Variable
from torch import optim # 경사 하강법을 이용해 가중치를 구하기 위한 옵티마이저 라이브러리

import torch.nn as nn
import torch.nn.functional as F
import os
import cv2
from tqdm import tqdm_notebook as tqdm # 진행 상황을 가시적으로 표현해주는데
import random
from matplotlib import pyplot as plt

device = torch.device("mps" if torch.backends.mps.is_available() else "cpu")
```

```
# 이미지 데이터셋 전처리
class ImageTransform():
    def __init__(self, resize, mean, std):
        self.data_transform = {
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize, scale = (0.5, 1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean, std)
            ])
        }
    def __call__(self, img, phase):
        return self.data_transform[phase](img)
```

1. `torchvision.transforms = transforms.Compose([transforms.RandomResizedCrop(resize, scale = (0.5, 1.0)), transform.RandomHorizontalFlip(), transforms.ToTensor(), transforms.Normalize(mean, std)])`
 - `transforms.Compose` : 이미지를 변형할 수 있는 방식들의 묶음
 - `transforms.RandomResizedCrop` : 입력 이미지를 주어진 크기로 조정, `scale` : 원래 이미지를 임의의 크기 (0.5~1.0)만큼 면적을 무작위로 자름
 - `transforms.RandomHorizontalFlip` : 주어진 확률로 이미지를 수평 반전시킴
 - 확률 지정 x: 기본값인 0.5의 확률로 이미지들이 수평 반전됨
 - 훈련 이미지 중 반은 위아래로 뒤집힌 상태로 두고, 반은 그대로 사용
 - `transforms.ToTensor`
 - `ImageFolder` 메서드를 비롯해서 `torchvision` method는 이미지를 읽을 때 파이썬 이미지 라이브러리인 PIL 사용
 - PIL을 사용해서 이미지를 읽으면 생성되는 이미지의 범위가 [0, 255]이며, 배열의 차원이 H * W * C로 표현됨

- 이후 효율적인 연산을 위해 `ToTensor()` 를 이용해 $C * H * W$ 으로 차원이 바뀌고, 픽셀 값의 범위도 $[0.0, 1.0]$ 사이가 됨
- `transforms.Normalize` : 전이 학습에서 사용하는 사전 훈련된 모델들은 대개 ImageNet 데이터셋에서 훈련됨. 따라서, 사전 훈련된 모델을 사용하기 위해서는 ImageNet 데이터의 **각 채널별 평균과 표준 편차에 맞는 정규화**가 필요함.
 - Normalize 메서드 안에 사용된 (mean: 0.485, 0.456, 0.406), (std: 0.229, 0.224, 0.225) 든 ImageNet 에서 이미지들의 RGB 채널마다 평균과 표준편차를 의미함
 - OpenCV를 사용해 이미지를 읽어온다면 RGB 이미지가 아닌 BGR 이미지이므로 채널 순서에 주의해야함

2. `__call__` 함수는 클래스를 호출하는 메서드.

- `__init__` 이 인스턴스 초기화를 위해 사용된다면, `__call__` 은 인스턴스가 호출되었을 때 실행함
- 클래스에 `__call__` 함수가 있을 경우 클래스 객체 자체를 호출하면 `__call__` 함수의 return 값이 반환됨

```
# 이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리
path = os.getcwd()
cat_directory = os.path.join(path, 'chap06/data/dogs-vs-cats/Cat/')
dog_directory = os.path.join(path, 'chap06/data/dogs-vs-cats/Dog/')

cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in
                               os.listdir(cat_directory)])
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in
                               os.listdir(dog_directory)])

# 개와 고양이 이미지들을 합쳐서 image_filepaths에 저장
images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400] # 400 개를 훈련 세트로
val_images_filepaths = correct_images_filepaths[400:-10] # 끝에서 10개 전까지를
test_images_filepaths = correct_images_filepaths[-10:] # 마지막 10개를 테스트 세트로
```

```
print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))
```

1. `cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in os.listdir(cat_directory)])`
 - `sorted` : 데이터를 정렬된 리스트로 만들어서 반환
 - `os.listdir` : 지정한 디렉터리 내 모든 파일의 리스트를 반환. 예제에서 사용하는 Cat 디렉터리의 이미지 파일들을 모두 반환함
2. `images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]` : *는 리스트의 요소를 Unpack 해서 하나의 리스트로 합침 → 고양이 + 강아지 전체 이미지 경로 리스트
3. `correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]`
 - OpenCV의 `cv2.imread()` 로 이미지 파일을 실제로 읽을 수 있는지 확인
 - 읽기에 실패한 이미지 파일 (`None`)은 제외 → 깨진 이미지 제거

테스트 데이터셋 이미지 확인 함수

```
def display_image_grid(images_filepaths, predicted_labels = (), cols = 5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows = rows, ncols = cols, figsize = (12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        true_label = os.path.normpath(image_filepath).split(os.sep)[-2]
        predicted_label = predicted_labels[i] if predicted_labels else true_label
        color = "green" if true_label == predicted_label else "red"
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(predicted_label, color = color)
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
```

1. `image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)`
 - `image` : 입력 이미지
 - `cv2.COLOR_BGR2RGB` : 변환할 이미지의 색상 지정 (BGR → RGB로 변경)
2. `true_label = os.path.normpath(image_filepath).split(os.sep)[-2]`
 - `os.path.normpath(image_filepath)` : 경로명 정규화

- A//B, A/B/, A/.B 및 A/./B 모두 A/B 로 경로를 통일
 - `split(os.sep)[-2]` : 경로를 / or \를 기준으로 분할할 때 사용
 - 예를 들어 `c:/temp/user/a.jpg` 라는 경로가 있을 때 `split(os.sep)`를 적용하면 `['c:', 'temp', 'user', 'a.jpg']` 처럼 분할됩니다. 또한, `split(os.sep)[-2]`를 적용하면 'user'를 반환할 것
 - 즉, 이미지 경로에서 **폴더 이름(=정답 라벨)** 추출
3. `predicted_label = predicted_labels[i] if predicted_labels else true_label`
- `predicted_label` 에 `predicted_labels` 의 값이 있으면 그 값을 사용하고, 없을 경우 `true_label` 을 사용

```
# 테스트 데이터셋 이미지 출력
display_image_grid(test_images_filepaths)
```



모델 학습

1. 데이터셋에 학습할 데이터의 경로를 정의하고 그 경로에서 데이터를 읽어 옴
2. 데이터셋 크기가 클 수 있으므로 `__init__` 에서 전체 데이터를 읽어 오는 것이 아니라 경로만 저장해놓고, `__getitem__` method에서는 이미지를 읽어 옴 (즉, 데이터를 어디에서 가져올지 결정)
3. 데이터로더에서 데이터셋의 데이터를 메모리로 불러오는데, 한꺼번에 전체 데이터를 불러오는 것이 아니라 배치 크기만큼 분할하여 가져옴

```

# 이미지 데이터셋 클래스 정의
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform = None, phase = 'train'):
        self.file_list = file_list
        # DogvsCatDataset 클래스를 호출할 때 transform에 대한 매개 변수를 받아옴
        self.transform = transform
        self.phase = phase #'train' 적용

    # images_filepaths 데이터셋의 전체 길이 반환
    def __len__(self):
        return len(self.file_list)

    # 데이터셋에서 데이터를 가져오는 부분으로 결과는 텐서 형태가 됨
    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path) # img_path 위치에서 이미지 데이터들을 가져옴
        img_transformed = self.transform(img, self.phase) # 이미지에 'train' 전처리
        label = img_path.split('/')[-1].split('.')[0]
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0
        return img_transformed, label

```

1. `label = img_path.split('/')[-1].split('.')[0]` : 이미지 데이터에 대한 레이블 값(dog, cat)을 가져옴
 - `img_path.split('/')[-1]` : 이미지의 전체 경로에서 '/'를 제거하고 label.number.jpg 형태를 가져옴
 - `.split('.')[0]` : label.number.jpg에서 .을 제거하고 label만 가져옴

```

# 이미지 데이터셋 정의
train_dataset = DogvsCatDataset(train_images_filepaths,
                                transform=ImageTransform(size, mean, std), phase = 'train')
val_dataset = DogvsCatDataset(val_images_filepaths,
                              transform=ImageTransform(size, mean, std), phase = 'val')

index = 0
# 훈련 데이터 (train_dataset.__getitem__[0][0]의 크기 size() 출력)
print(train_dataset.__getitem__(index)[0].size())

```

```
# 훈련 데이터의 레이블 출력
print(train_dataset.__getitem__(index)[1])
```

```
torch.Size([3, 224, 224])
0
```

- 이미지는 컬러 상태에서 224 * 224 크기를 가지며 레이블이 0으로 출력됨.
- 즉, 훈련 데이터셋의 레이블이 0을 갖기 때문에 '고양이' 이미지가 포함되어있다는 것을 유추할 수 있음

```
# 데이터로더 정의
```

```
train_dataloader = DataLoader(train_dataset, batch_size= batch_size, shuffle = True)
val_dataloader = DataLoader(val_dataset, batch_size= batch_size, shuffle = False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}
```

```
batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)
```

```
torch.Size([32, 3, 224, 224])
tensor([1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0,
        0, 0, 0, 1, 0, 0, 0, 0])
```

```
# 모델의 네트워크 클래스
```

```
class LeNet(nn.Module):
```

```
    def __init__(self):
```

```
        super(LeNet, self).__init__()
```

```
        # 2D 합성곱층 적용
```

```
        # 입력: (3, 224, 224)
```

```
        # 출력: (16, 220, 220) (weight-kernel_size + 1)/stride; 224-5+1 = 220
```

```
        self.cnn1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=5,
                               stride = 5, padding = 0)
```

```
        # 음수는 0으로 만들고 나머지는 그대로 통과
```

```
        # (16, 220, 220)
```

```
        self.relu1 = nn.ReLU()
```

```

# 최대 풀링층 (2 * 2 영역에서 최댓값 하나만 선택) -> (16, 110, 110)
self.maxpool1 = nn.MaxPool2d(kernel_size=2)
# 입력: (16, 110, 110)
# 출력: (32, 106, 106); 110 - 5 + 1
self.cnn2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=5,
                      stride = 1, padding = 0)
# (32, 106, 106)
self.relu2 = nn.ReLU()
# 최대 풀링층 (2 * 2 영역에서 최댓값 하나만 선택) -> (32, 53, 53)
self.maxpool2 = nn.MaxPool2d(kernel_size=2)

# 완전 연결층, 일렬로 펼쳐서 완전 연결층으로 넣음
# 전체 입력 뉴런 수: 32 * 53 * 53
# 출력: 512
self.fc1 = nn.Linear(32*53*53, 512)
self.relu5 = nn.ReLU()
# 입력 : 512
# 출력: 2개의 노드
self.fc2 = nn.Linear(512, 2)
# dim = 1; 각 샘플의 열 방향으로 softmax 적용
self.output = nn.Softmax(dim = 1)

def forward(self, x):
    out = self.cnn1(x)
    out = self.relu1(out)
    out = self.maxpool1(out)
    out = self.cnn2(out)
    out = self.relu2(out)
    out = self.maxpool2(out)
    out = self.maxpool2(out)
    out = out.view(out.size(0), -1) # 완전 연결층에 데이터를 전달하기 위해 데이터
    out = self.fc1(out)
    out = self.fc2(out)
    out = self.output(out)
    return out

```

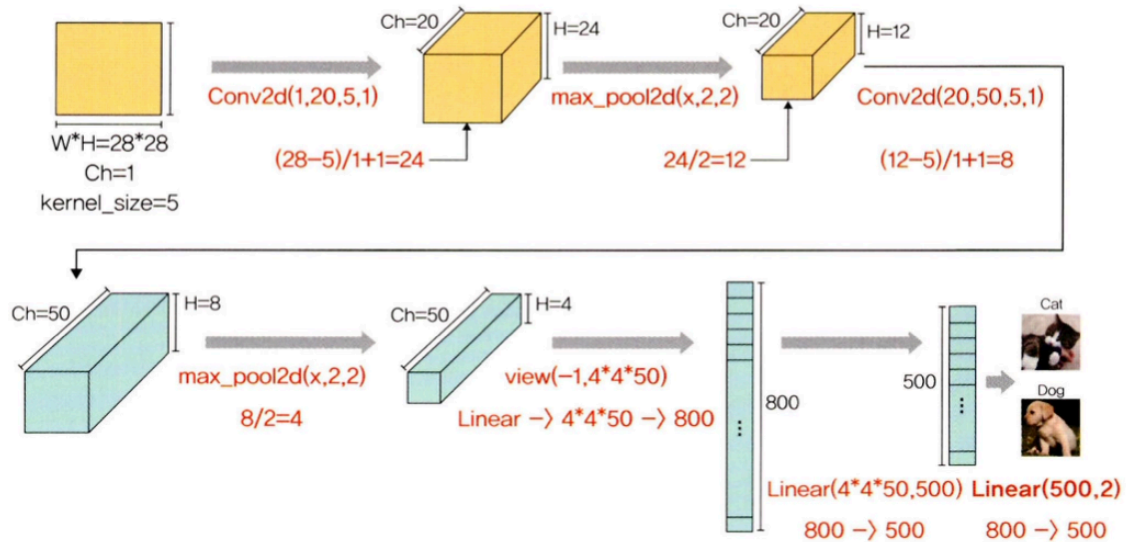
Conv2d 계층에서 출력 크기를 구하는 공식

- 출력 크기 = $(W - F + 2P)/S + 1$

MaxPool2d 계층에서의 출력 크기 구하는 공식

- 출력 크기 = IF/F

▼ 그림 6-5 합성곱층과 풀링층의 크기(형태) 계산



pip install torchsummary

- 케라스와 같은 형태로 모델을 출력해볼 수 있는 라이브러리

```
from torchsummary import summary
summary(model, input_size = (3, 224, 224))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 16, 220, 220]	1,216
ReLU-2	[-1, 16, 220, 220]	0
MaxPool2d-3	[-1, 16, 110, 110]	0
Conv2d-4	[-1, 32, 106, 106]	12,832
ReLU-5	[-1, 32, 106, 106]	0
MaxPool2d-6	[-1, 32, 53, 53]	0
Linear-7	[-1, 512]	46,023,168
Linear-8	[-1, 2]	1,026
Softmax-9	[-1, 2]	0

Total params: 46,038,242
 Trainable params: 46,038,242
 Non-trainable params: 0

Input size (MB): 0.57
 Forward/backward pass size (MB): 19.47
 Params size (MB): 175.62
 Estimated Total Size (MB): 195.67

- 네트워크 내 파라미터 수와 구조가 이해하기 쉽게 표현되어 있음

```
# 학습 가능한 파라미터 수 확인
def count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'The model has {count_parameters(model):,} trainable_parameters')
```

The model has 46,038,242 trainable_parameters

- 46,036,242 개의 파라미터를 학습해야하기 때문에 이미지 전체가 아닌 일부 이미지만 예제에서 사용

```
# 옵티마이저와 손실 함수 정의
optimizer = optim.SGD(model.parameters(), lr = 0.001, momentum=0.9)
criterion = nn.CrossEntropyLoss()
```

1. 경사 하강법으로 모멘텀 SGD 사용

- 모멘텀 SGD는 SGD에 관성이 추가된 것으로 매번 기울기를 구하지만, 가중치를 수정하기 전에는 이전 수정 방향 (+,-)을 참고하여 같은 방향으로 일정한 비율만 수정되게 하는 방법

- `optim.SGD(model.parameters(), lr=0.001, momentum=0.9)`
 - `model.parameters()` : 경사 하강법을 통해 궁극적으로 업데이트하고자하는 파라미터는 **가중치와 바이어스**이다.
 - `lr` : 학습률, 가중치를 변경할 때 얼마나 크게 변경할지 결정
 - `momentum` : SGD를 적절한 방향으로 가속화하며 흔들림을 줄여주는 매개변수

```
model = model.to(device)
criterion = criterion.to(device)
```

모델의 학습 함수 정의

```
import time
```

```
def train_model(model, dataloader_dict, criterion, optimizer, num_epoch):
```

```
    since = time.time()
```

```
    best_acc = 0.0
```

```
    for epoch in range(num_epoch): # epoch를 10으로 설정했으므로 10회 반복
```

```
        print(f'Epoch {epoch + 1}/{num_epoch}')
```

```
        print('-' * 20)
```

```
        for phase in ['train', 'val']:
```

```
            if phase == 'train':
```

```
                model.train() # 모델을 학습시키겠다는 의미
```

```
            else:
```

```
                model.eval()
```

```
            epoch_loss = 0.0
```

```
            epoch_corrects = 0
```

```
            # 이때 dataloader_dict는 훈련 데이터셋
```

```
            for inputs, labels in tqdm(dataloader_dict[phase]):
```

```
                inputs = inputs.to(device) # 훈련 데이터셋 CPU에 할당
```

```
                labels = labels.to(device)
```

```
                optimizer.zero_grad() # 역전파 단계 실행 전 기울기를 0으로 초기화
```

```
            with torch.set_grad_enabled(phase == 'train'):
```

```
                outputs = model(inputs)
```

```
                _, preds = torch.max(outputs, 1)
```

```
                loss = criterion(outputs, labels) # 손실 함수를 이용한 오차 계산
```

```

        if phase == 'train':
            loss.backward() # 모델의 학습 가능한 모든 파라미터에 대해 기울기 계
            optimizer.step() # optimizer의 step 함수를 호출하면 파라미터를 갱신
            epoch_loss += loss.item() * inputs.size(0)
            epoch_corrects += torch.sum(preds == labels.data) # 정답과 예측이
            epoch_loss = epoch_loss / len(dataloader_dict[phase].dataset) # 최종 손
            epoch_acc = epoch_corrects.float() / len(dataloader_dict[phase].dataset)

        print(f'{phase}: Loss: {epoch_loss:.4f} ACC: {epoch_acc:.4f}')

        if phase == 'val' and epoch_acc > best_acc: # 검증 데이터셋에 대한 가장
            best_acc = epoch_acc
            best_models_wts = model.state_dict()

    time_elapsed = time.time() - since
    print(f'Training complete in {time_elapsed // 60 :.0f}m {time_elapsed % 60:.
    print(f'Best val Acc: {best_acc:.4f}')
    return model

```

1. `epoch_loss += loss.item() * inputs.size(0)` : 오차 * 입력

- 왜 오차와입력을 곱하는 가? → 손실함수의 `reduction` parameter
- `reduction` 파라미터의 기본값은 `mean` . 즉, 정답과 예측 값의 오차를 구한 후 그 값들의 평균을 반환
- 손실 함수의 특성상 전체 오차를 배치 크기로 나눔으로써 평균을 반환하기 때문에 `epoch_loss` 를 계산하는 동안 `loss.item()` 과 `input.size(0)` 을 곱해준다.
- **현재 배치(batch)의 손실 값을 전체 데이터셋에 대한 평균 손실로 환산하기 위해 사용**

좀 더 쉽게 설명하면:

- `loss.item()` 은 현재 배치에 대한 평균 손실 값이야.
- 하지만 우리는 전체 에폭(epoch) 동안의 총 손실을 알고 싶잖아?
- 그래서 그 평균 손실에 **현재 배치의 샘플 수 (`inputs.size(0)`)**를 곱해줘.
이건 쉽게 말하면, "이 손실 값이 이만큼의 데이터에 대해 계산된 거야" 라고 알려주는 거지.

예를 들어서:

- 어떤 배치에서 손실값이 `0.5` 이고, 배치 크기가 `32` 라면
→ $0.5 * 32 = 16$
→ 그 배치는 총 16의 손실을 만든 거야.

이걸 모든 배치마다 반복해서 더한 후,

마지막에 `epoch_loss / len(dataset)` 하면 전체 평균 손실이 계산되는 구조야.

```
import time
num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)
```

Epoch 9/10

100%  13/13 [00:03<00:00, 3.80it/s]

train: Loss: 0.6413 ACC: 0.6200

100%  3/3 [00:00<00:00, 6.46it/s]

val: Loss: 0.6817 ACC: 0.5435

Epoch 10/10

100%  13/13 [00:04<00:00, 3.16it/s]

train: Loss: 0.6440 ACC: 0.6500

100%  3/3 [00:00<00:00, 4.84it/s]

val: Loss: 0.6781 ACC: 0.5652

Training complete in 0m 54

Best val Acc: 0.5652

- 검증 데이터셋을 이용한 모델 학습 결과 최고 56%의 정확도를 보이고 있지만, 그닥 높지 않은 결과이다.

모델 테스트

```
# 모델 테스트를 위한 함수 정의
import pandas as pd
```

```
id_list = []
pred_list = []
_id = 0
```

```
with torch.no_grad():
```

```
    # 역전파 중 텐서들에 대한 변화도를 계산할 필요가 없음을 나타냄 (훈련 데이터셋 모델)
```

```
    for test_path in tqdm(test_images_filepaths): # 테스트 데이터셋 이용
```

```
        img = Image.open(test_path)
```

```
        _id = test_path.split('/')[-1].split('.')[1]
```

```
        transform = ImageTransform(size, mean, std)
```

```
        img = transform(img, phase = 'val') # 테스트 데이터셋 전처리 적용
```

```
        img = img.unsqueeze(0)
```

```
        img = img.to(device)
```

```
        model.eval()
```

```
        outputs = model(img)
```

```
        preds = F.softmax(outputs, dim = 1)[:,1].tolist()
```

```
        id_list.append(_id)
```

```
        pred_list.append(preds[0])
```

```
res = pd.DataFrame({
```

```
    'id': id_list,
```

```
    'label': pred_list
```

```
})
```

```
res.sort_values(by = 'id', inplace=True)
```

```
res.reset_index(drop=True, inplace=True)
```

```
res.to_csv('chap06/data/LeNet', index = False) # 데이터 프레임을 csv 파일로 저장
```

1. `torch.unsqueeze(0)` 는 텐서에 차원을 추가할 때 사용함

- (0)은 차원이 추가될 위치(dim)를 나타냄

```
import torch
a = torch.tensor([[1,2],[3,4]])
print(a.shape)
print(a.squeeze().shape)
```

```
torch.Size([2, 2])
torch.Size([2, 2])
```

```
b = a.squeeze()
print(b)
print(b.unsqueeze(0))
print(b.unsqueeze(0).shape)
print('-'*5)
print(b.unsqueeze(1))
print(b.unsqueeze(1).shape)
print('-'*5)
print(b.unsqueeze(2))
print(b.unsqueeze(2).shape)
```

```
tensor([[1, 2],
        [3, 4]])
tensor([[[1, 2],
         [3, 4]]])
torch.Size([1, 2, 2])
-----
tensor([[[1, 2],
         [3, 4]]])
torch.Size([2, 1, 2])
-----
tensor([[[1],
         [2]],
        [[3],
         [4]]])
torch.Size([2, 2, 1])
```

2. `preds = F.softmax(output, dim =1)[:1].tolist()`

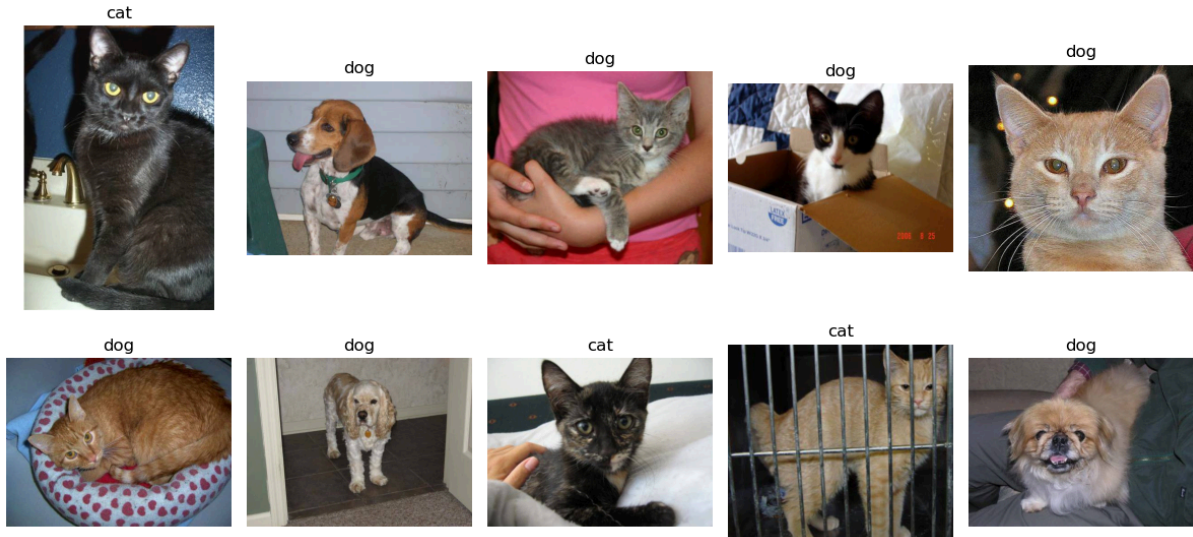
- softmax는 지정된 차원을 따라 텐서의 요소가 (0,1) 범위에 있고 합계가 1이 되도록 크기를 조정함
- `outputs` 에 softmax를 적용해 각 **행**의 합이 1이 되도록함
- `[:,1]` : softmax를 적용한 값 중 모든 행(row)에서 2 번째 칼럼을 가져옴
- `.tolist()` 로 배열을 리스트 형태로 변환함

```
# 테스트 데이터셋의 예측 결과 호출
res.head(10)
```

	id	label
0	109	0.544831
1	145	0.403272
2	15	0.674348
3	162	0.519995
4	167	0.522378
5	200	0.555786
6	210	0.685970
7	211	0.617568
8	213	0.403682
9	224	0.669067

```
# 테스트 데이터셋 이미지를 출력하기 위한 함수 정의
class_ = classes = {0: 'cat', 1: 'dog'} # cat, dog에 대한 클래스 정의
def display_image_grid(images_filepaths, predicted_labels=(), cols = 5):
    rows = len(images_filepaths) // cols
    figure, ax = plt.subplots(nrows = rows, ncols = cols, figsize = (12, 6))
    for i, image_filepath in enumerate(images_filepaths):
        image = cv2.imread(image_filepath)
        image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
        a = random.choice(res['id'].values) # 데이터 프레임의 id라는 칼럼에서 임의로
        label = res.loc[res['id'] == a, 'label'].values[0]
        if label > 0.5: # 레이블 값이 0.5보다 크면 개
            label = 1
        else: # 레이블 값이 0.5보다 작다면 고양이
            label = 0
        ax.ravel()[i].imshow(image)
        ax.ravel()[i].set_title(class_[label])
        ax.ravel()[i].set_axis_off()
    plt.tight_layout()
    plt.show()
```

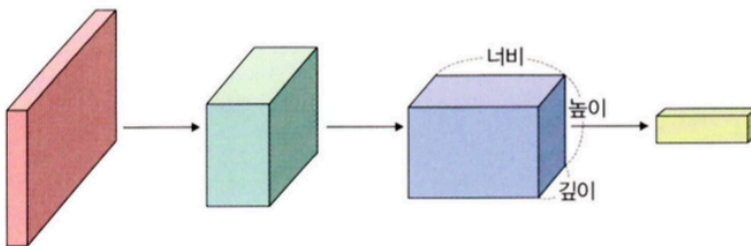
테스트 데이터셋 예측 결과 이미지 출력
`display_image_grid(test_images_filepaths)`



예측력이 그닥 좋지 않음. 이는 kaggle 전체 데이터가 아닌 일부 데이터만 사용했기 때문이다. 전체 데이터를 사용한다면 예측력은 좋아지겠지만, 훈련 시간이 며칠 씩 걸릴 수도 있음

6.1.2 AlexNet

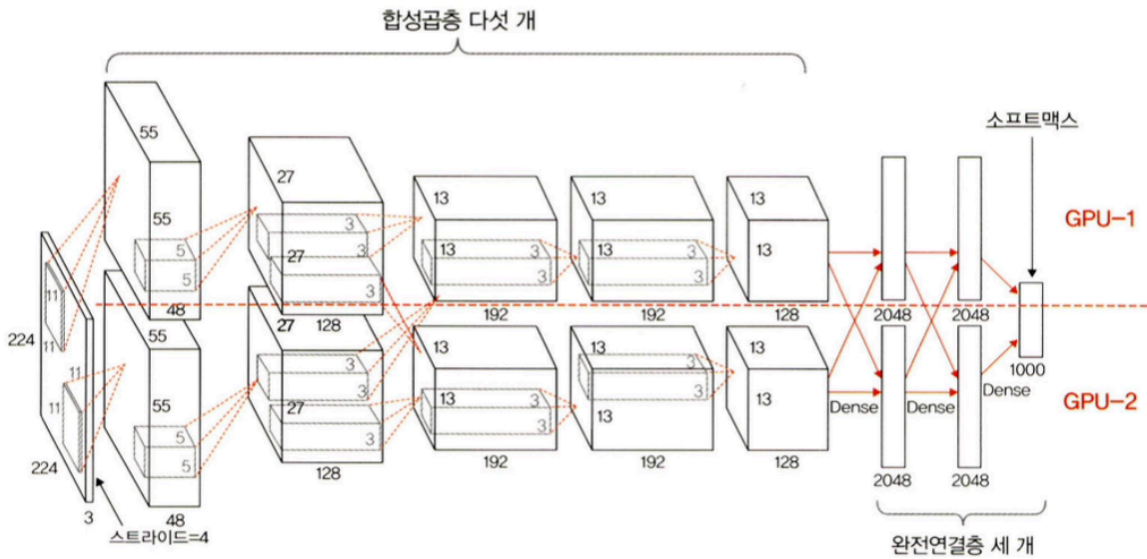
▼ 그림 6-8 CNN 구조



- CNN은 기본적으로 이미지를 다루기 때문에 W (너비) * H (높이) * C (채널, 깊이)의 3차원 구조를 가짐
- 처음에는 R, G, B로 깊이가 3으로 시작하지만, 합성곱을 거치면서 특성 맵이 만들어지고 이것에 따라 중간 영상의 깊이가 달라짐

- **AlexNet**은 합성곱층 총 5 개와 완전 연결층 3개로 구성되어 있으며, 맨 마지막 완전 연결층은 카테고리 1000개를 분류하기 위해 **소프트맥스** 활성화 함수를 사용함. 활성화 함수는 ReLU를 사용
- 전체적으로 보면 GPU 2 개를 기반으로 한 병렬 구조인 점을 제외하면 LeNet-5와 크게 다르지 않다

♥ 그림 6-9 AlexNet 구조

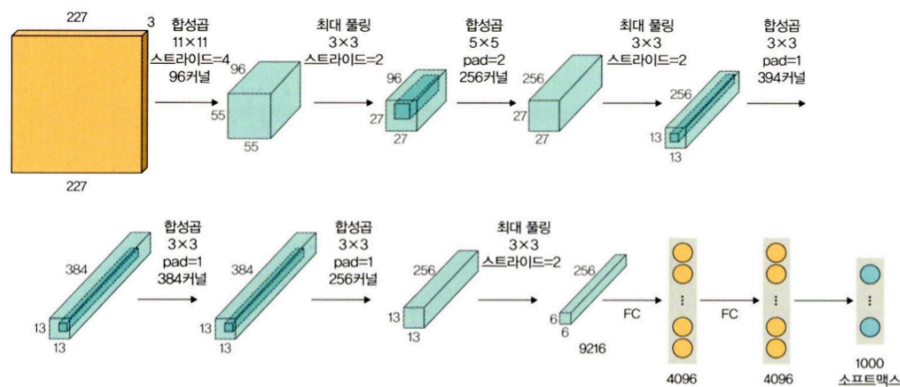


♥ 표 6-2 AlexNet 구조 상세

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	227×227	—	—	—
합성곱층	96	55×55	11×11	4	렐루(ReLU)
최대 풀링층	96	27×27	3×3	2	—
합성곱층	256	27×27	5×5	1	렐루(ReLU)
최대 풀링층	256	13×13	3×3	2	—
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	384	13×13	3×3	1	렐루(ReLU)
합성곱층	256	13×13	3×3	1	렐루(ReLU)
최대 풀링층	256	6×6	3×3	2	—
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	4096	—	—	렐루(ReLU)
완전연결층	—	1000	—	—	소프트맥스(softmax)

- 네트워크에는 총 6600만 개의 학습 가능한 변수가 있다. 네트워크에 대한 입력은 $227 * 227 * 3$ 크기의 RGB 이미지이며, 각 클래스에 해당하는 $1000 * 1$ 확률 벡터를 출력함
- AlexNet의 첫 번째 합성곱층 커널의 크기는 $11 * 11 * 3$ 이며, 스트라이드를 4로 적용하여 특성맵을 96 개 생성 $\rightarrow 55 * 55 * 96$ 의 출력을 가짐 $(227 - 11 + 0) / 4 + 1 = 55$
- 첫 번째 계층을 거치면서 GPU-1에서는 주로 컬러와 상관 없는 정보를 추출하기 위한 커널이 학습되고, GPU-2에서는 주로 컬러와 관련된 정보를 추출하기 위한 커널이 학습됨

▼ 그림 6-11 AlexNet 예제 네트워크



- 예제에서는 마지막 계층에 클래스 2개만 사용
- LeNet에서 정의한 이미지 전처리 함수와 데이터 분리 코드 사용
-

필요한 라이브러리 호출

```
import torch
```

```
import torchvision
```

```
from torch.utils.data import DataLoader, Dataset
```

```
from torchvision import transforms # 이미지 변환 (전처리) 기능을 제공하는 라이브러리
```

```
from torch.autograd import Variable
```

```
from torch import optim # 경사 하강법을 이용해 가중치를 구하기 위한 옵티마이저 라이브러리
```

```
import torch.nn as nn
```

```
import torch.nn.functional as F
```

```
import os
```

```
import cv2
```

```
from PIL import Image
```

```
from tqdm import tqdm_notebook as tqdm # 진행 상황을 가시적으로 표현해주는데
```

```
device = torch.device("cpu")
```

- device를 "mps"로 설정하면 AdaptiveAvgPool2d에서 input/output 크기가 안 맞아서 MPS에서는 에러 발생

→ CPU를 사용

```
# 이미지 데이터셋 전처리
```

```
class ImageTransform():
```

```
    def __init__(self, resize, mean, std):
```

```
        self.data_transform = {
```

```
            'train': transforms.Compose([
```

```
                transforms.RandomResizedCrop(resize, scale = (0.5, 1.0)),
```

```
                transforms.RandomHorizontalFlip(),
```

```
                transforms.ToTensor(),
```

```
                transforms.Normalize(mean, std)
```

```
            ]),
```

```
            'val': transforms.Compose([
```

```
                transforms.Resize(256),
```

```
                transforms.CenterCrop(resize),
```

```
                transforms.ToTensor(),
```

```
                transforms.Normalize(mean, std)
```

```
            ])
```

```
        }
```

```
    def __call__(self, img, phase):
```

```
        return self.data_transform[phase](img)
```

```
# 이미지 데이터셋을 불러온 후 훈련, 검증, 테스트로 분리
```

```
path = os.getcwd()
```

```
cat_directory = os.path.join(path, 'chap06/data/dogs-vs-cats/Cat/')
```

```
dog_directory = os.path.join(path, 'chap06/data/dogs-vs-cats/Dog/')
```

```
cat_images_filepaths = sorted([os.path.join(cat_directory, f) for f in  
                                os.listdir(cat_directory)])
```

```
dog_images_filepaths = sorted([os.path.join(dog_directory, f) for f in  
                                os.listdir(dog_directory)])
```

```
# 개와 고양이 이미지들을 합쳐서 image_filepaths에 저장
```

```

images_filepaths = [*cat_images_filepaths, *dog_images_filepaths]
correct_images_filepaths = [i for i in images_filepaths if cv2.imread(i) is not None]

random.seed(42)
random.shuffle(correct_images_filepaths)
train_images_filepaths = correct_images_filepaths[:400]
val_images_filepaths = correct_images_filepaths[400:-10]
test_images_filepaths = correct_images_filepaths[-10:]

print(len(train_images_filepaths), len(val_images_filepaths), len(test_images_filepaths))

```

```

# 이미지 데이터셋 클래스 정의
class DogvsCatDataset(Dataset):
    def __init__(self, file_list, transform = None, phase = 'train'):
        self.file_list = file_list
        # DogvsCatDataset 클래스를 호출할 때 transform에 대한 매개 변수를 받아옴
        self.transform = transform
        self.phase = phase # 'train' 적용

    # images_filepaths 데이터셋의 전체 길이 반환
    def __len__(self):
        return len(self.file_list)

    # 데이터셋에서 데이터를 가져오는 부분으로 결과는 텐서 형태가 됨
    def __getitem__(self, idx):
        img_path = self.file_list[idx]
        img = Image.open(img_path) # img_path 위치에서 이미지 데이터들을 가져옴
        img_transformed = self.transform(img, self.phase) # 이미지에 'train' 전처리
        label = img_path.split('/')[-1].split('.')[0]
        if label == 'dog':
            label = 1
        elif label == 'cat':
            label = 0
        return img_transformed, label

# 변수값 정의
size = 256 # AlexNet은 깊이가 깊은 네트워크를 사용하므로 이미지 크기가 256이 아님
mean = (0.485, 0.456, 0.406)

```

```

std = (0.229, 0.224, 0.225)
batch_size = 32

# 이미지 데이터셋 정의
train_dataset = DogvsCatDataset(train_images_filepaths,
                                transform=ImageTransform(size, mean, std), phase = 'train')
val_dataset = DogvsCatDataset(val_images_filepaths,
                              transform=ImageTransform(size, mean, std), phase = 'val')

index = 0
# 훈련 데이터 (train_dataset.__getitem__[0][0]의 크기 size() 출력)
print(train_dataset.__getitem__(index)[0].size())
# 훈련 데이터의 레이블 출력
print(train_dataset.__getitem__(index)[1])

```

```

torch.Size([3, 256, 256])
0

```

```

# 데이터로더 정의
train_dataloader = DataLoader(train_dataset, batch_size= batch_size, shuffle = True)
val_dataloader = DataLoader(val_dataset, batch_size= batch_size, shuffle = False)
dataloader_dict = {'train': train_dataloader, 'val': val_dataloader}

batch_iterator = iter(train_dataloader)
inputs, label = next(batch_iterator)
print(inputs.size())
print(label)

```

```

torch.Size([32, 3, 256, 256])
tensor([0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0,
        0, 0, 1, 1, 1, 1, 0, 1])

```

```

# AlexNet 모델 네트워크 정의
class AlexNet(nn.Module):
    def __init__(self) → None:
        super(AlexNet, self).__init__()
        self.features = nn.Sequential(

```

```

nn.Conv2d(3, 64, kernel_size=11, stride=4, padding = 2),
nn.ReLU(inplace= True),
nn.MaxPool2d(kernel_size=3, stride=2),
nn.Conv2d(64, 192, kernel_size=5, padding = 2),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=3, stride=2),
nn.Conv2d(192, 384, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.Conv2d(384, 256, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.Conv2d(256, 256, kernel_size=3, padding=1),
nn.ReLU(inplace=True),
nn.MaxPool2d(kernel_size=3, stride = 2)
)
self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
self.classifier = nn.Sequential(
    nn.Dropout(),
    nn.Linear(256 * 6 * 6, 4096),
    nn.ReLU(inplace = True),
    nn.Dropout(),
    nn.Linear(4096, 512),
    nn.ReLU(inplace=True),
    nn.Linear(512, 2),
)
def forward(self, x: torch.Tensor) → torch.Tensor:
    x = self.features(x)
    x = self.avgpool(x)
    x = torch.flatten(x, 1)
    x = self.classifier(x)
    return x

```

1. ReLU 함수의 `inplace = True` : 연산에 대한 결과값을 새로운 변수에 저장하는 것이 아닌 기존 데이터를 대체하는 것
 - 기존 값을 연산 결과값으로 대체함으로써 기존 값들을 무시하겠다
2. `nn.AdaptiveAvgPool2d` 는 `nn.AvgPool2d` 처럼 풀링을 위해 사용함
 - `AvgPool2d` 에서는 풀링에 대한 커널 및 스트라이드 크기를 정의해야 동작함

- `nn.AvgPool2d((3, 2), stride = (2, 1))` 처럼 지정하면 (N, C, H_{in}, W_{in}) 크기의 입력을 (N, C, H_{out}, W_{out}) 크기로 출력
 - $H_{out} = \left\lfloor \frac{H_{in} + 2 * padding[0] - kernel_size[0]}{stride[0]} \right\rfloor + 1$
 - $W_{out} = \left\lfloor \frac{W_{in} + 2 * padding[0] - kernel_size[0]}{stride[1]} \right\rfloor + 1$
- `AdaptiveAvgPool2d` 는 풀링 작업이 끝날 때 필요한 출력 크기를 정의함. 즉, 출력에 대한 크기만 정하기 때문에 출력 크기에 대한 조정이 상당히 쉬워짐

```
# 모델 객체 생성
model = AlexNet()
```

```
# optimizer, loss function
optimizer = optim.SGD(model.parameters(), lr = 0.001, momentum = 0.9)
criterion = nn.CrossEntropyLoss()
```

```
# 모델 네트워크 구조 확인
from torchsummary import summary
summary(model, input_size = (3, 256, 256))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 63, 63]	23,296
ReLU-2	[-1, 64, 63, 63]	0
MaxPool2d-3	[-1, 64, 31, 31]	0
Conv2d-4	[-1, 192, 31, 31]	307,392
ReLU-5	[-1, 192, 31, 31]	0
MaxPool2d-6	[-1, 192, 15, 15]	0
Conv2d-7	[-1, 384, 15, 15]	663,936
ReLU-8	[-1, 384, 15, 15]	0
Conv2d-9	[-1, 256, 15, 15]	884,992
ReLU-10	[-1, 256, 15, 15]	0
Conv2d-11	[-1, 256, 15, 15]	590,080
ReLU-12	[-1, 256, 15, 15]	0
MaxPool2d-13	[-1, 256, 7, 7]	0
AdaptiveAvgPool2d-14	[-1, 256, 6, 6]	0
Dropout-15	[-1, 9216]	0
Linear-16	[-1, 4096]	37,752,832
ReLU-17	[-1, 4096]	0
Dropout-18	[-1, 4096]	0
Linear-19	[-1, 512]	2,097,664
ReLU-20	[-1, 512]	0
Linear-21	[-1, 2]	1,026

...

Forward/backward pass size (MB): 10.90
Params size (MB): 161.44
Estimated Total Size (MB): 173.10

```
model = model.to(device)
criterion = criterion.to(device)
```

```
# 모델 학습
num_epoch = 10
model = train_model(model, dataloader_dict, criterion, optimizer, num_epoch)
```

```
# 모델 테스트를 위한 함수 정의
import pandas as pd

id_list = []
pred_list = []
_id = 0
with torch.no_grad():
    # 역전파 중 텐서들에 대한 변화도를 계산할 필요가 없음을 나타냄 (훈련 데이터셋 모델
    for test_path in tqdm(test_images_filepaths): # 테스트 데이터셋 이용
        img = Image.open(test_path)
        _id = test_path.split('/')[-1].split('.')[1]
```

```

transform = ImageTransform(size, mean, std)
img = transform(img, phase = 'val') # 테스트 데이터셋에 검증용 전처리 적용
img = img.unsqueeze(0)
img = img.to(device)

model.eval()
outputs = model(img)
preds = F.softmax(outputs, dim = 1)[:1].tolist()

id_list.append(_id)
pred_list.append(preds[0])

res = pd.DataFrame({
    'id': id_list,
    'label': pred_list
})

res.to_csv('chap06/data/LeNet', index = False) # 데이터 프레임을 csv 파일로 저장

```

```

# 데이터 프레임 결과 확인
res.head(10)

```

	id	label
0	145	0.501116
1	211	0.501349
2	162	0.500936
3	200	0.502314
4	210	0.500876
5	224	0.500851
6	213	0.500591
7	109	0.501931
8	15	0.501104
9	167	0.500676

```

# 예측 결과를 시각적으로 표현하기 위한 함수 정의
class_ = classes = {0: 'cat', 1: 'dog'} # cat, dog에 대한 클래스 정의
def display_image_grid(images_filepaths, predicted_labels=(), cols = 5):

```



```

rows = len(images_filepaths) // cols
figure, ax = plt.subplots(nrows = rows, ncols = cols, figsize = (12, 6))
for i, image_filepath in enumerate(images_filepaths):
    image = cv2.imread(image_filepath)
    image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

    a = random.choice(res['id'].values) # 데이터 프레임의 id라는 칼럼에서 임의로
    label = res.loc[res['id'] == a, 'label'].values[0]
    if label > 0.5: # 레이블 값이 0.5보다 크면 개
        label = 1
    else: # 레이블 값이 0.5보다 작다면 고양이
        label = 0
    ax.ravel()[i].imshow(image)
    ax.ravel()[i].set_title(class_[label])
    ax.ravel()[i].set_axis_off()
plt.tight_layout()
plt.show()

```

```
display_image_grid(test_images_filepaths)
```



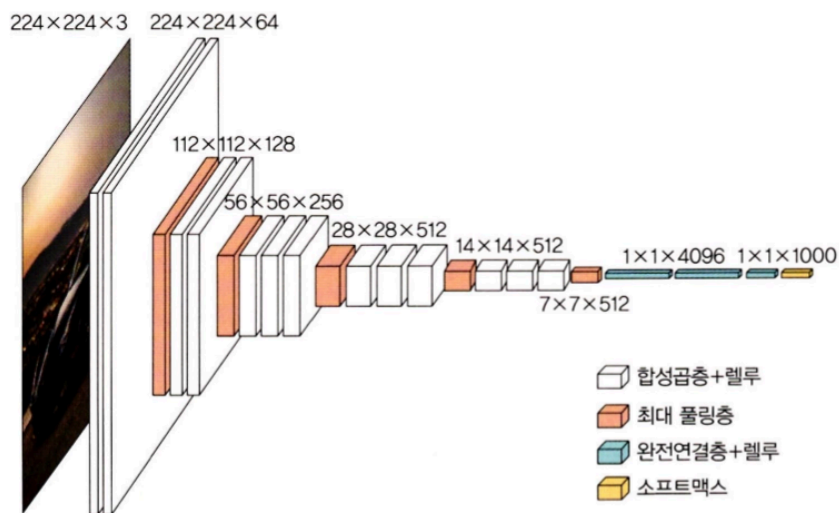
역시나 작은 양의 데이터만 사용했기 때문에 예측 성능이 좋지 않음

6.1.3 VGGNet

- 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 등장

- 깊이의 영향만 확인하고자 합성곱층에서 사용하는 필터/커널의 크기를 가장 작은 3×3 으로 고정
- 네트워크 계층의 총 개수에 따라 VGG16, VGG19 등 다양한 유형의 VGGNet이 있다.
- **VGG16**에는 파라미터가 총 1억 3300만 개 존재한다. 이때, 모든 합성곱 커널의 크기는 3×3 , 최대 풀링 커널의 크기는 2×2 , 스트라이드는 2이다. 결과적으로 $224 \times 224 \times 64$ 가 생성된다. 마지막 16번째 계층을 제외하고는 모두 ReLU 활성화 함수가 적용됨

▼ 그림 6-13 VGG16 구조



계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
이미지	1	224×224	–	–	–
합성곱층	64	224×224	3×3	1	렐루(ReLU)
합성곱층	64	224×224	3×3	1	렐루(ReLU)
최대 풀링층	64	112×112	2×2	2	–
합성곱층	128	112×112	3×3	1	렐루(ReLU)
합성곱층	128	112×112	3×3	1	렐루(ReLU)
최대 풀링층	128	56×56	2×2	2	–
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
합성곱층	256	56×56	3×3	1	렐루(ReLU)
최대 풀링층	256	28×28	2×2	2	–
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
합성곱층	512	28×28	3×3	1	렐루(ReLU)
최대 풀링층	512	14×14	2×2	2	–

계층 유형	특성 맵	크기	커널 크기	스트라이드	활성화 함수
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
합성곱층	512	14×14	3×3	1	렐루(ReLU)
최대 풀링층	512	7×7	2×2	2	–
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	4096	–	–	렐루(ReLU)
완전연결층	–	1000	–	–	소프트맥스(softmax)

```
# 필요한 라이브러리 호출
import copy # 객체 복사
import numpy as np
```

```

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
import torch.utils.data as data
import torchvision
import torchvision.transforms as transforms
import torchvision.datasets as Datasets

device = torch.device('mps' if torch.backends.mps.is_available() else 'cpu')

```

1. `import copy`

- 단순한 객체 복사

```

original = [1, 2, 3] ----- original이라는 변수에 [1, 2, 3] 저장
copy_o = original ----- copy_o에 original 복사(shallow copy)
print(copy_o)

```

```

copy_o[2] = 10 ----- copy_o의 3을 10으로 값을 변경
print(copy_o)
print(original)

```

original 값을 복사한 copy_o에서 3 값을 10으로 바꾼 결과는 다음과 같습니다.

```

[1, 2, 3]
[1, 2, 10]
[1, 2, 10]

```

copy_o뿐만 아니라 원래 값인 original의 3도 10으로 바뀌었습니다.

- 얕은 복사: `copy.copy()`

```
import copy

original = [[1, 2], 3]
copy_o = copy.copy(original) ----- original 값을 copy_o에 얹은 복사(copy.copy())
print(copy_o)
copy_o[0] = 100 ----- copy_o의 [1, 2] 값을 100으로 변경
print(copy_o)
print(original)

append = copy.copy(original)
append[0].append(4) ----- 첫 번째 리스트([1, 2])에 4를 추가
print(append)
print(original)
```

- 깊은 복사: `copy.deepcopy()`

`copy.copy()` 는 얹은 복사를 수행하고, `copy.deepcopy()` 는 깊은 복사를 수행합니다.

- 얹은 복사 (`copy.copy()`): 객체의 복사본을 생성하지만, 객체 내부에 포함된 중첩된 객체 (리스트, 딕셔너리 등)는 원본 객체와 참조를 공유합니다. 중첩된 객체를 수정하면 복사본과 원본 모두에 영향을 줄 수 있습니다.
- 깊은 복사 (`copy.deepcopy()`): 객체의 모든 요소를 재귀적으로 복사하여 독립적인 복사본을 만듭니다. 중첩된 객체도 새롭게 생성되므로 복사본을 수정하더라도 원본에 영향을 미치지 않습니다.

예제:

```
import copy

# 원본 리스트
original = [[1, 2], [3, 4]]

# 얹은 복사
shallow_copy = copy.copy(original)

# 깊은 복사
deep_copy = copy.deepcopy(original)

# 중첩된 리스트 수정
original[0][0] = 99

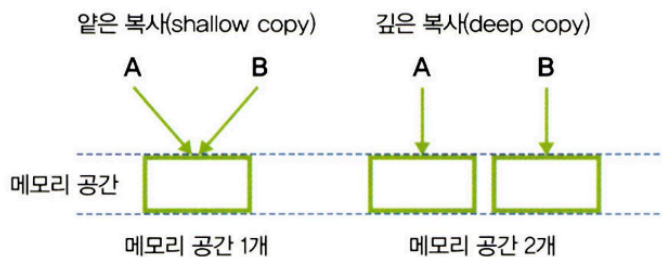
print("원본:", original)      # [[99, 2], [3, 4]]
```

```
print("얕은 복사:", shallow_copy) # [[99, 2], [3, 4]]
print("깊은 복사:", deep_copy) # [[1, 2], [3, 4]]
```

위 예제에서 볼 수 있듯이, 얕은 복사는 중첩된 객체를 원본과 공유하기 때문에 원본을 수정하면 얕은 복사본에도 영향을 미칩니다. 반면, 깊은 복사는 중첩된 객체까지 별도로 복사하기 때문에 원본과 독립적으로 동작합니다.

단순 복사는 완전히 동일하게 복사되지만, 얕은 복사와 깊은 복사는 다음과 같은 차이가 있습니다.

▼ 그림 6-14 얕은 복사와 깊은 복사



모델 유형 정의

숫자 (output channel, 출력 채널): Conv2d 수행

출력 채널이 다음 계층의 입력 채널이 됨

M: 최대 풀링을 수행

합성곱층 8 + 풀링층 3 = 전체 계층 11

vgg11_config = [64, 'M', 128, 'M', 256, 256, 'M', 512, 512, 'M']

합성곱층 10 + 풀링층 3 = 전체 계층 13

vgg13_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 'M', 512, 512, 'M', 512, 512]

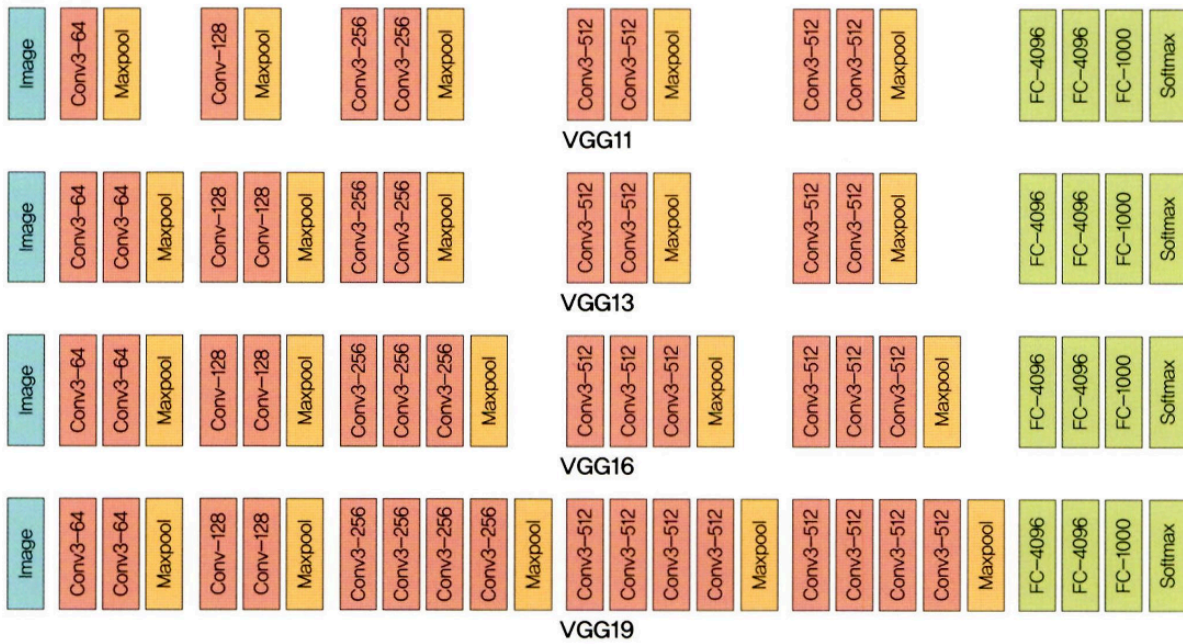
합성곱층 13 + 풀링층 3 = 전체 계층 16

vgg16_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 'M']

합성곱층 16 + 풀링층 3 = 전체 계층 19

vgg19_config = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 256, 'M', 512, 512, 512, 'M', 512, 512, 512, 'M']

▼ 그림 6-15 VGG의 다양한 모델에 대한 네트워크



VGG 계층 정의

```
def get_vgg_layers(config, batch_norm):
```

```
    layers = []
```

```
    in_channels = 3
```

```
    for c in config: # vgg11_config의 값들을 가져옴
```

```
        assert c == 'M' or isinstance(c, int)
```

```
        if c == 'M': # 불러온 값이 M이라면 최대 풀링 적용
```

```
            layers += [nn.MaxPool2d(kernel_size=2)]
```

```
        else: # 불러온 값이 숫자이면 합성곱 적용
```

```
            conv2d = nn.Conv2d(in_channels, c, kernel_size=3, padding = 1)
```

```
            if batch_norm: # 배치 정규화를 적용할지에 대한 코드
```

```
                # 배치 정규화가 적용될 경우 배치 정규화 + ReLU 사용
```

```
                layers += [conv2d, nn.BatchNorm2d(c), nn.ReLU(inplace=True)]
```

```
            else:
```

```
                # 배치 정규화가 적용되지 않을 경우 ReLU만 적용
```

```
                layers += [conv2d, nn.ReLU(inplace=True)]
```

```
            in_channels = c
```

```
    return nn.Sequential(*layers) # 네트워크의 모든 계층 반환
```

1. `assert c == 'M' or isinstance(c, int)` : 조건문 정의

- `assert c == 'M'` 에서 가정설정문 `assert`는 뒤의 조건이 True가 아니면 에러를 발생 시킴. 즉. `c == 'M'`이 아닌 경우 에러 발생

```
-----
AssertionError                                Traceback (most recent call last)
<ipython-input-1-b211624433f8> in <module>
      1 a = 1
----> 2 assert a == 11

AssertionError:
```

- `isinstance` 는 주어진 조건이 True인지 판단

```
print(isinstance(1, int)) ----- 1이 integer인지 판단
print(isinstance(1.2, int)) ----- 1.2가 integer인지 판단
print(isinstance('deep learning', str)) ----- deep learning이 string인지 판단
```

다음은 `isinstance`로 조건이 True/False인지 확인한 결과입니다.

```
True
False
True
```

- 즉, `c != 'M'`이거나 `c`가 `int`가 아닐 경우 오류 발생

모델 계층 생성 (배치 정규화 계층 추가)

```
vgg11_layers = get_vgg_layers(vgg11_config, batch_norm=True)
```

1. `batch_norm(Batch Normalization)` 은 데이터 평균을 0으로, 표준 편차를 1로 분포시키는 것.
 - 각 계층에서 입력 데이터의 분포는 **앞 계층에서 업데이트된 가중치에 따라 변화**
 - 즉, 각 계층마다 변화되는 분포는 학습 속도를 늦출 뿐만 아니라 학습도 어렵게 함
 - 따라서 각 계층의 입력에 대한 분산을 평균 0, 표준 편차 1로 분포 시키는 것이 `batch_norm` 이다.

```
print(vgg11_layers)
```



```

Sequential(
  (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (2): ReLU(inplace=True)
  (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (6): ReLU(inplace=True)
  (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (10): ReLU(inplace=True)
  (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (13): ReLU(inplace=True)
  (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (17): ReLU(inplace=True)
  (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (20): ReLU(inplace=True)
  (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  ...
  (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (27): ReLU(inplace=True)
  (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
)

```

- 배치 정규화가 추가된 것 확인 가능

```

OUTPUT_dim = 2
model = VGG(vgg11_layers, OUTPUT_dim)
print(model)

```

```

VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): ReLU(inplace=True)
    (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (17): ReLU(inplace=True)
    (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (20): ReLU(inplace=True)
    (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (classifier): Sequential(
    (0): Linear(in_features=25088, out_features=4096, bias=True)
    (1): ReLU(inplace=True)
    (2): Dropout(p=0.5, inplace=False)
    (3): Linear(in_features=4096, out_features=4096, bias=True)
    (4): ReLU(inplace=True)
    (5): Dropout(p=0.5, inplace=False)
    (6): Linear(in_features=4096, out_features=2, bias=True)
  )
)

```

- 앞서 정의했던 vgg11_layers, 완전 연결층과 출력층이 합쳐짐

- VGG는 사전 훈련된 모델
- 즉, 이미 누군가가 대용량의 이미지 데이터로 학습 시켰으며, 최상의 상태로 튜닝을 거쳐 모든 사람이 사용할 수 있도록 공유한 사전 훈련된 모델이다

```

import torchvision.models as models
pretrained_model = models.vgg11_bn(pretrained = True)
print(pretrained_model)

```

1. `models.vgg11_bn(pretrained = True)`

- `models.vgg11_bn` : VGG11 기본 모델에 배치 정규화가 적용된 모델을 사용하겠다는 의미
- `pretrained = True` : 사전 훈련된 모델을 사용하겠다 (미리 학습된 파라미터 값 사용)

```

VGG(
  (features): Sequential(
    (0): Conv2d(3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU(inplace=True)
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (4): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (5): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU(inplace=True)
    (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (8): Conv2d(128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (9): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (10): ReLU(inplace=True)
    (11): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (12): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (13): ReLU(inplace=True)
    (14): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (15): Conv2d(256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (16): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (17): ReLU(inplace=True)
    (18): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (19): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (20): ReLU(inplace=True)
    (21): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (22): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (23): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (24): ReLU(inplace=True)
    (25): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))

    (26): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (27): ReLU(inplace=True)
    (28): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(7, 7))
  (classifier): Sequential(
    (0): Linear(in_features=25088, out_features=4096, bias=True)
    (1): ReLU(inplace=True)
    (2): Dropout(p=0.5, inplace=False)
    (3): Linear(in_features=4096, out_features=4096, bias=True)
    (4): ReLU(inplace=True)
    (5): Dropout(p=0.5, inplace=False)
    (6): Linear(in_features=4096, out_features=1000, bias=True)
  )
)

```

이미지 데이터 전처리

```

train_transforms = transforms.Compose([
    transforms.Resize((256, 256)), # 이미지를 256 * 256 크기로 조정
    transforms.RandomRotation(5), # 5도 이하로 이미지 회전
    transforms.RandomHorizontalFlip(0.5),
    transforms.ToTensor(),
    transforms.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.
])

```

```

test_transforms = transforms.Compose([
    transforms.Resize((256, 256)), # 이미지를 256 * 256 크기로 조정
    transforms.ToTensor(),
    transforms.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.
])

```

```

# ImageFolder를 이용해 데이터셋 불러오기
train_path = os.path.join(path, 'chap06/data/catanddog/train/')
test_path = os.path.join(path, 'chap06/data/catanddog/test/')

train_dataset = torchvision.datasets.ImageFolder(
    train_path,
    transform = train_transforms
)
# ImageFolder는 계층적인 폴더 구조를 가지고 있는 데이터셋을 불러올 때 사용
# train 폴더 하위에 Cat, Dog 폴더가 계층적으로 위치하고 있기 때문에 ImageFolder 사용

test_dataset = torchvision.datasets.ImageFolder(
    test_path,
    transform = test_transforms
)

print(len(train_dataset)), print(len(test_dataset))

```

```

529
12

(None, None)

```

```

# 훈련과 검증 데이터 분할
VALID_RATIO = 0.9
# 전체 훈련 데이터 중 90%를 훈련 데이터셋으로 사용
n_train_example = int(len(train_dataset) * VALID_RATIO)
# 전체 훈련 데이터 중 남은 10%를 검증 데이터셋으로 사용
n_valid_example = int(len(train_dataset) - n_train_example)

train_data, valid_data = data.random_split(train_dataset,
                                           [n_train_example, n_valid_example])

```

1. `random_split(train_dataset, [n_train_example, n_valid_example])` : 훈련과 검증 데이터셋을 나누는 용도로 사용. 데이터가 데이터로더로 넘어간 이후에는 분리가 불가능하므로 데이터셋 단계에서 분리해야함

- `train_dataset` : 분할에 사용될 데이터셋
- `[n_train_example, n_valid_example]` : 훈련과 검증 데이터셋의 크기 지정 ([훈련 데이터셋 크기, 검증 데이터셋 크기])

```
# 검증 데이터 전처리
valid_data = copy.deepcopy(valid_data)
valid_data.dataset.transform = test_transforms
```

왜 `valid_data`를 `deepcopy`하는가?

- PyTorch의 `random_split()` 은 원래 데이터셋의 일부를 참조하는 `Subset` 객체를 반환합니다.
 - 즉, `train_data` , `valid_data` 는 원래의 `train_dataset` 을 공유하고 있는 `Subset` 입니다.
 - 이 상태에서 `valid_data.dataset.transform` 을 바꾸면, `train_data.dataset.transform` 에도 영향을 줄 수 있습니다.
- **transform 설정은 원본 `dataset` 에 적용됩니다.**
 - 만약 `valid_data` 의 transform만 바꾸고 싶다면, `train_data` 와 `valid_data` 가 서로 독립적인 **dataset 객체를 가져야** 합니다.
- 🙌 그래서 `copy.deepcopy()` 로 완전히 새로운 `valid_data` 객체를 만들어서,
 - `valid_data.dataset.transform` 을 수정해도,
 - `train_data.dataset.transform` 에는 영향을 주지 않도록 한 것입니다.

```
# 훈련, 검증, 테스트 데이터셋 수 확인
print(f'Number of Training Example: {len(train_data)}')
print(f'Number of Validation Example: {len(valid_data)}')
print(f'Number of Testing Example: {len(test_dataset)}')
```

```
Number of Training Example: 476
Number of Validation Example: 53
Number of Testing Example: 12
```

```
# 메모리로 데이터 불러오기
BATCH_SIZE = 128
# 훈련 데이터셋은 임의로 섞어서 가져옴
train_iterator = data.DataLoader(train_data, shuffle = True,
```

```

        batch_size = BATCH_SIZE)
valid_iterator = data.DataLoader(valid_data,
        batch_size = BATCH_SIZE)
test_iterator = data.DataLoader(test_dataset, batch_size=BATCH_SIZE)

```

```

# 모델 학습에서 사용할 옵티마이저와 손실 함수 정의
optimizer = optim.Adam(model.parameters(), lr = 1e-7)
criterion = nn.CrossEntropyLoss()

```

```

model = model.to(device)
criterion = criterion.to(device)

```

```

# 모델 정확도 측정 함수
def calculate_accuracy(y_pred, y):
    top_pred = y_pred.argmax(1, keepdim = True)
    correct = top_pred.eq(y.view_as(top_pred)).sum()
    acc = correct.float() / y.shape[0]
    return acc

```

1. `correct = top_pred.eq(y.view_as(top_pred)).sum()`

- `eq` : equal의 약자로 서로 같은지를 비교
- `view_as(other)` : other의 텐서 크기를 사용하겠다. 즉, y에 대한 텐서 크기를 `top_pred`의 텐서 크기로 변경하겠다
- `.sum()` : 합계를 구하는 것으로, 예측(`top_pred`)과 정답(`y`)이 일치하는 것들의 개수를 합산하겠다

```

# 모델 학습 함수 정의

```

```

def train(model, iterator, optimizer, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.train()

    for (x,y) in iterator: # epoch를 10으로 설정했으므로 10회 반복
        x = x.to(device)

```

```

y = y.to(device)

optimizer.zero_grad()
y_pred, _ = model(x)
loss = criterion(y_pred, y)
acc = calculate_accuracy(y_pred, y)
loss.backward()
optimizer.step()

epoch_loss += loss.item()
epoch_acc += acc.item()

return epoch_loss / len(iterator), epoch_acc / len(iterator)

```

```

# 모델 성능 측정 함수
def evaluate(model, iterator, criterion, device):
    epoch_loss = 0
    epoch_acc = 0

    model.eval()
    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y = y.to(device)
            y_pred, _ = model(x)
            loss = criterion(y_pred, y)
            acc = calculate_accuracy(y_pred, y)
            epoch_loss += loss.item()
            epoch_acc += acc.item()
    return epoch_loss / len(iterator), epoch_acc / len(iterator)

```

```

# 학습 시간 측정 함수
def epoch_time(start_time, end_time):
    elapsed_time = end_time - start_time
    elapsed_mins = int(elapsed_time / 60)

```

```
elapsed_secs = int(elapsed_time - (elapsed_mins * 60))
return elapsed_mins, elapsed_secs
```

```
# 모델 학습
EPOCHS = 5
best_valid_loss = float('inf')
for epoch in range(EPOCHS):
    start_time = time.monotonic()
    # 훈련 데이터셋을 모델에 적용한 결과(오차와 정확도)를 train_loss와 train_acc에 저장
    train_loss, train_acc = train(model, train_iterator, optimizer, criterion, device)
    # 검증 데이터셋을 모델에 적용한 결과(오차와 정확도)를 valid_loss와 valid_acc에 저장
    valid_loss, valid_acc = evaluate(model, valid_iterator, criterion, device)

    # valid_loss가 가장 작은 값을 구하고 그 상태의 모델을 VGG-model.pt로 저장
    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), 'chap06/data/VGG-model.pt')

    end_time = time.monotonic()
    epoch_mins, epoch_secs = epoch_time(start_time, end_time) # 모델 훈련 시작

    print(f'Epoch: {epoch+1:02} | Epoch Time: {epoch_mins}m {epoch_secs}s')
    print(f'\tTrain Loss: {train_loss:.3f} | Train Acc: {train_acc * 100:.2f}%')
    print(f'\tValid Loss: {valid_loss:.3f} | Valid Acc: {valid_acc * 100:.2f}%')
```

```
Epoch: 01 | Epoch Time: 3m 34s
    Train Loss: 0.694 | Train Acc: 50.93%
    Valid Loss: 0.693 | Valid Acc: 52.83%
Epoch: 02 | Epoch Time: 2m 54s
    Train Loss: 0.695 | Train Acc: 51.59%
    Valid Loss: 0.693 | Valid Acc: 52.83%
Epoch: 03 | Epoch Time: 2m 59s
    Train Loss: 0.691 | Train Acc: 52.51%
    Valid Loss: 0.692 | Valid Acc: 54.72%
Epoch: 04 | Epoch Time: 3m 29s
    Train Loss: 0.688 | Train Acc: 50.66%
    Valid Loss: 0.691 | Valid Acc: 64.15%
Epoch: 05 | Epoch Time: 3m 26s
    Train Loss: 0.702 | Train Acc: 50.35%
    Valid Loss: 0.691 | Valid Acc: 73.58%
```


- 성능이 나쁘진 않지만, 엄청 좋지도 않음

```
# 테스트 데이터셋을 이용한 모델 성능 측정
model.load_state_dict(torch.load('chap06/data/VGG-model.pt'))
test_loss, test_acc = evaluate(model, test_iterator, criterion, device)
print(f'Test Loss: {test_loss:.3f} | Test Acc: {test_acc * 100:.2f}')
```

Test Loss: 0.695 | Test Acc: 41.67%

- 역시나 결과가 좋지 않음 (테스트 데이터셋을 늘리면 성능이 좋아질 것)

```
# 테스트 데이터셋을 이용한 모델의 예측 확인 함수
def get_predictions(model, iterator):
    model.eval()
    images = []
    labels = []
    probs = []

    with torch.no_grad():
        for (x, y) in iterator:
            x = x.to(device)
            y_pred, _ = model(x)
            y_prob = F.softmax(y_pred, dim = 1)
            top_pred = y_prob.argmax(1, keepdim = True)
            images.append(x.cpu())
            labels.append(y.cpu())
            probs.append(y_prob.cpu())

    # 앞에서 정의한 get_predictions() 함수의 반환값을 각각 images, labels, probs에
    images = torch.cat(images, dim = 0)
    labels = torch.cat(labels, dim = 0)
    probs = torch.cat(probs, dim = 0)
    return images, labels, probs
```

1. `argmax(1, keepdim = True)`

- `argmax` : 최댓값을 갖는 **인덱스**를 반환
 - 텐서를 **1D로 flatten**한 후, 최댓값의 **전체 인덱스** 반환

- `max`: 최댓값을 반환

```
x = torch.rand(5, 3)
print(x)
```

✓ 0.0s

```
tensor([[0.4219, 0.2352, 0.9376],
        [0.0169, 0.6476, 0.6907],
        [0.4546, 0.8701, 0.1650],
        [0.8327, 0.5630, 0.4773],
        [0.8026, 0.7897, 0.8786]])
```

```
print(torch.max(x)), print(torch.argmax(x))
```

✓ 0.0s

```
tensor(0.9376)
tensor(2)
```

- `1`: 행 (axis = 0), 열 (axis = 1)을 따라 가장 큰 값의 index를 찾음

▼ 그림 6-17 차원(dim)

	Col1	Col2	Col3
Row1			
Row2			
Row3			

axis=0(dim=0)

axis=1(dim=1)

- `keepdim = True`: 출력 텐서를 입력과 동일한 크기로 유지
2. `torch.cat`: 텐서를 연결할 때 사용
- `dim = 0`: 행 방향으로 텐서 연결
 - `dim = 1`: 열 방향으로 텐서 연결

```

x = torch.Tensor([[1, 2, 3], [2, 3, 4]]) # 2 * 3
y = torch.Tensor([[4, 5, 6], [5, 6, 7]]) # 2 * 3

print(x.shape)
print('-' * 10)
print(torch.cat([x], dim = 0)) # 4 * 3
print('-' * 10)
print(torch.cat([x, y])) # 4 * 3
print('-' * 10)
print(torch.cat([x, y], dim = 0)) # 4 * 3
print('-' * 10)
print(torch.cat([x, y], dim = 1)) # 2 * 6

torch.Size([2, 3])
-----
tensor([[1., 2., 3.],
        [2., 3., 4.]])
-----
tensor([[1., 2., 3.],
        [2., 3., 4.],
        [4., 5., 6.],
        [5., 6., 7.]])
-----
tensor([[1., 2., 3.],
        [2., 3., 4.],
        [4., 5., 6.],
        [5., 6., 7.]])
-----
tensor([[1., 2., 3., 4., 5., 6.],
        [2., 3., 4., 5., 6., 7.]])

```

예측 중에서 정확하게 예측한 것 추출

```
images, labels, probs = get_predictions(model, test_iterator)
```

```
pred_labels = torch.argmax(probs, 1)
```

```
corrects = torch.eq(labels, pred_labels) # 예측과 정답이 같은지 비교
```

```
correct_examples = []
```

```
for image, label, prob, correct in zip(images, labels, probs, corrects):
```

```
    if correct:
```

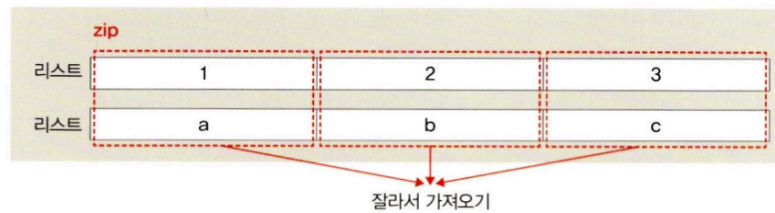
```
        correct_examples.append((image, label, prob))
```

reverse = True: 내림차순으로 정렬

```
correct_examples.sort(reverse=True, key = lambda x: torch.max(x[2], dim = 0))
```

1. `zip()`: 여러 개의 리스트 혹은 튜플을 합쳐서 새로운 튜플 타입으로 반환

- 튜플은 순서가 있는 데이터의 집합으로 순서가 있기 때문에 인덱스를 이용한 데이터 접근이 가능하지만, 한 번 생성되면 값을 변경할 수 없음



```
a = [1, 2, 3]
b = ['a', 'b', 'c']

for x, y in zip(a, b):
    print(x, y)
```

✓ 0.0s

```
1 a
2 b
3 c
```

2. `sort(key)` : 데이터를 정렬할 때 `key` 값을 가지고 정렬하며 기본값은 오름차순이다. `(reverse = True)` 로 내림차순 변경

- `lambda` : 정의할 필요가 없는 일회용 함수

▼ 그림 6-19 일반 함수와 람다 함수

```
def 함수명(매개변수):
    return 반환될 결과값

lambda 매개변수 : 반환될 결과값
```

- `torch.max(x, dim = 0).values` : `x`의 행 방향 최댓값들의 `value`를 가지고 온다 (`.values` 가 없으면 `values`와 `indices`가 함께 출력된다)

```
x = torch.randn([4, 4])
print(x)
print(torch.max(x, dim = 0))
```

✓ 0.0s

```
tensor([[ 1.1232, -0.5377,  0.4002, -0.1315],
        [ 1.9097,  0.4819,  0.6404, -0.9489],
        [ 0.6542,  0.5701, -1.5338,  0.4796],
        [ 1.3774,  0.9358,  1.6200, -0.3793]])
torch.return_types.max(
  values=tensor([1.9097, 0.9358, 1.6200, 0.4796]),
  indices=tensor([1, 3, 3, 2]))
```

- 앞에서 이미지의 크기 및 전처리 과정 등을 거쳤기 때문에 본래 이미지 색상이 왜곡됨
→ 본래 이미지로 돌려주기 위한 전처리 필요

♥ 그림 6-20 본래 이미지 색상이 왜곡됨



♥ 그림 6-21 본래 이미지



```
# 본래 이미지 출력을 위한 전처리
def normalize_image(image):
    image_min = image.min()
    image_max = image.max()
    image.clamp_(min = image_min, max = image_max)
    image.add_(-image_min).div_(image_max - image_min + 1e-5)
    return image
```

1. `clamp_()` 는 텐서의 값을 지정한 범위로 잘라주는 함수로, _가 붙었으므로 **in-place** 연산 (기존의 메모리에 위치한 값을 새로운 값으로 대체함)
 - `image_min` 보다 작으면 `image_min` 으로, `image_max` 보다 크면 `image_max` 로 잘라줌
2. `torch.add` : 더하는 method

1. `image.add_(-image_min)` → 이미지 전체에서 최솟값을 뺍니다.

예:

$$x' = x - \min(x)$$

2. `.div_(image_max - image_min + 1e-5)` → 위 결과를 최댓값-최솟값으로 나눕니다.

예:

$$x'' = \frac{x'}{\max(x) - \min(x) + \epsilon}$$

결과적으로, 이미지의 픽셀 값이 0~1 사이로 정규화됩니다.

여기서 `1e-5` 는 0으로 나누는 것을 방지하기 위한 **안전한 작은 수(ϵ)**입니다.

모델이 정확하게 예측한 이미지 출력 함수

```
def plot_most_correct(correct, classes, n_images, normalize = True):
    rows = int(np.sqrt(n_images))
    cols = int(np.sqrt(n_images))
    fig = plt.figure(figsize = (25, 20))
    for i in range(rows * cols):
        ax = fig.add_subplot(rows, cols, i + 1) # 출력하려는 그래프 개수만큼 subplot
        image, true_label, probs = correct[i]
        image = image.permute(1, 2, 0)

        true_prob = probs[true_label]
        correct_prob, correct_label = torch.max(probs, dim = 0)
        true_class = classes[true_label]
        correct_class = classes[correct_label]

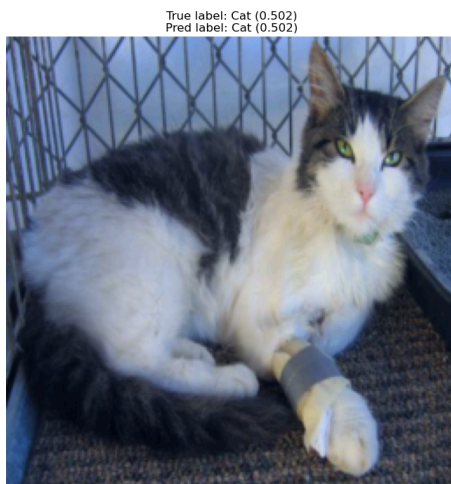
        if normalize:
            image = normalize_image(image)

        ax.imshow(image.cpu().numpy())
        ax.set_title(f'True label: {true_class} ({true_prob:.3f})\n' \
                    f'Pred label: {correct_class} ({correct_prob:.3f})')
        ax.axis('off')
    fig.subplots_adjust(hspace = 0.4)
```

1. `image.permute` : 축을 변경할 때 사용

- `image` 는 보통 `C x H x W` 형태인데, matplotlib는 `H x W x C` 를 원하므로 `permute(1, 2, 0)` 으로 순서 변경.

```
classes = test_dataset.classes
N_IMAGES = 5
plot_most_correct(correct_examples, classes, N_IMAGES)
```



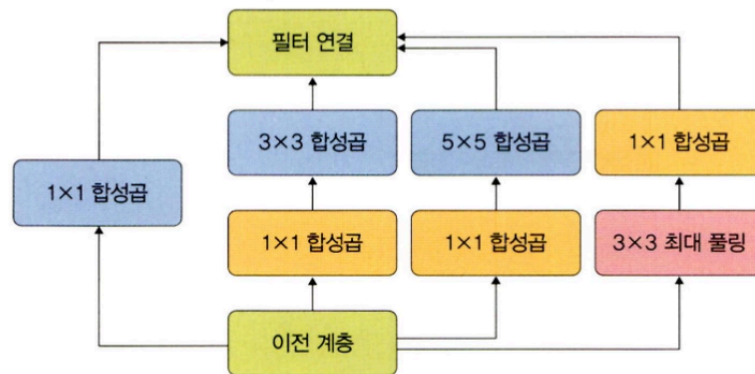
역시 데이터의 크기가 작아 정확하지 않다

6.1.4 GoogLeNet

- 주어진 하드웨어 자원을 최대한 효율적으로 사용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망이다.

- 깊고 넓은 신경망을 위해 GoogLeNet은 **inception module**을 추가함
 - 인셉션 모듈의 네 가지 연산
 - 1×1 합성곱
 - 1×1 합성곱 + 3×3 합성곱
 - 1×1 합성곱 + 5×5 합성곱
 - 3×3 최대 풀링(maxpooling) + 1×1 합성곱(convolutional)
- 인셉션 모듈에서는 특징을 효율적으로 추출하기 위해 1×1 , 3×3 , 5×5 합성곱 연산을 각각 수행한다. 3×3 최대 풀링은 입력과 출력의 높이와 너비가 같아야하므로 풀링 연산에서는 드물게 패딩을 추가해야한다.
- 결과적으로 GoogLeNet에 적용된 해결방법은 **희소 연결(sparse connectivity)**이다 .
- **희소 연결**: CNN은 합성곱, 풀링, 완전 연결층들이 서로 밀집하게 연결되어 있는데, 뻘뻘하게 연결된 신경망 대신 관련성이 높은 노드끼리만 연결하는 방법
 - 연산량을 줄어 과적합도 해결 가능

▼ 그림 6-23 GoogLeNet의 인셉션 모듈



- 딥러닝을 이용해 ImageNet과 같은 대회에 참여하거나 서비스를 제공하려면 대용량의 데이터를 학습해야함.
- 심층 신경망의 아키텍처에서 계층이 넓고 깊으면 인식률은 좋아지지만, 과적합이나 기울기 소멸 문제를 비롯한 학습 시간 지연과 연산 속도 등의 문제가 존재함. 특히 CNN에서 이런 문제들이 자주 나타나기 때문에 GoogLeNet (or Inception)이 그런 문제의 해결책으로 제시됨